HDOC-72

Dynamic Student Attendance Matrix Using Pointers

C Programming - Dynamic Memory Allocation

Problem Statement

Develop a C program to manage student attendance using a dynamically resizable matrix. Each row represents one student and each column represents one working day. Use 1 for Present and 0 for Absent. The program must use pointers for matrix handling and must not use a fixed-size 2D array.

- Accept the initial number of students and working days.
- Use malloc() to dynamically allocate memory for the initial attendance matrix.
- Accept the attendance details of the initial students and validate that only 0 or 1 is entered.
- Allow additional students to be added dynamically.
- Use realloc() to increase the matrix size when new students are added.
- Calculate and display the attendance percentage of each student.
- Use separate functions for creating the matrix, recording attendance, adding students, displaying records, calculating percentages, and releasing memory.
- Handle invalid input and memory-allocation failure appropriately, and free all dynamically allocated memory before termination.

Algorithm / Logic

1. Read the initial number of students and working days. Reject non-positive values.

2. Allocate an array of row pointers using malloc(). Allocate each student row dynamically for the required number of days.

3. Record attendance for every student. Re-prompt whenever the entered value is not 0 or 1.

4. Display the initial attendance matrix.

5. Read the number of additional students. Resize the row-pointer array using realloc().

6. Allocate memory for each newly added student row and record the new attendance values.

7. Display the resized attendance matrix.

8. For each student, count the number of 1s and compute: Attendance % = (Present Days / Working Days) x 100.

9. Free every row and finally free the row-pointer array.

Functions Used

Function	Purpose
createMatrix()	Allocates the initial dynamic attendance matrix using malloc().
recordAttendance()	Accepts and validates 0/1 attendance values.
displayAttendance()	Displays all student attendance records.
addStudents()	Expands the matrix with realloc() and allocates new rows.
calculatePercentages()	Computes and displays attendance percentage for each student.
freeMatrix()	Releases all dynamically allocated memory.

Header Files Used

- <stdio.h> - input/output operations.
- <stdlib.h> - malloc(), realloc(), free() and EXIT_FAILURE.

Complete C Program

```
#include <stdio.h>
#include <stdlib.h>

int **createMatrix(int students, int days) {
    int **matrix = (int **)malloc(students * sizeof(int *));
    if (matrix == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < students; i++) {
        matrix[i] = (int *)malloc(days * sizeof(int));
        if (matrix[i] == NULL) {
            printf("Memory allocation failed.\n");
            for (int j = 0; j < i; j++)
                free(matrix[j]);
            free(matrix);
            exit(EXIT_FAILURE);
        }
    }
    return matrix;
}

void recordAttendance(int **matrix, int start, int end, int days) {
    for (int i = start; i < end; i++) {
        printf("Enter attendance for Student %d (0/1):\n", i + 1);
        for (int j = 0; j < days; j++) {
            while (1) {
                printf("  Day %d: ", j + 1);
                if (scanf("%d", &matrix[i][j]) == 1 &&
                    (matrix[i][j] == 0 || matrix[i][j] == 1))
                    break;
```

```
            printf("  Invalid input! Enter only 0 or 1.\n");
            while (getchar() != '\n');
        }
      }
    }
}

void displayAttendance(int **matrix, int students, int days) {
    printf("\nStudent");
    for (int j = 0; j < days; j++)
        printf("\tDay %d", j + 1);
    printf("\n");

    for (int i = 0; i < students; i++) {
        printf("Student %d", i + 1);
        for (int j = 0; j < days; j++)
            printf("\t%d", matrix[i][j]);
        printf("\n");
    }
}

int **addStudents(int **matrix, int oldCount, int newCount, int days) {
    int total = oldCount + newCount;
    int **temp = (int **)realloc(matrix, total * sizeof(int *));

    if (temp == NULL) {
        printf("Memory reallocation failed.\n");
        for (int i = 0; i < oldCount; i++)
            free(matrix[i]);
        free(matrix);
        exit(EXIT_FAILURE);
    }
    matrix = temp;

    for (int i = oldCount; i < total; i++) {
        matrix[i] = (int *)malloc(days * sizeof(int));
        if (matrix[i] == NULL) {
            printf("Memory allocation failed.\n");
            for (int j = 0; j < i; j++)
                free(matrix[j]);
            free(matrix);
            exit(EXIT_FAILURE);
        }
    }
    return matrix;
}

void calculatePercentages(int **matrix, int students, int days) {
    printf("\nAttendance Percentage:\n");
    for (int i = 0; i < students; i++) {
        int present = 0;
        for (int j = 0; j < days; j++)
            present += matrix[i][j];
        printf("Student %d: %.2f%%\n", i + 1,
               (present * 100.0) / days);
    }
}

void freeMatrix(int **matrix, int students) {
    for (int i = 0; i < students; i++)
```

```
        free(matrix[i]);
    free(matrix);
}

int main(void) {
    int students, days, additional;

    printf("Enter initial number of students: ");
    if (scanf("%d", &students) != 1 || students <= 0) {
        printf("Invalid number of students.\n");
        return 1;
    }

    printf("Enter number of working days: ");
    if (scanf("%d", &days) != 1 || days <= 0) {
        printf("Invalid number of working days.\n");
        return 1;
    }

    int **attendance = createMatrix(students, days);
    recordAttendance(attendance, 0, students, days);

    printf("\nInitial Attendance Matrix:\n");
    displayAttendance(attendance, students, days);

    printf("\nEnter number of additional students: ");
    if (scanf("%d", &additional) != 1 || additional < 0) {
        printf("Invalid number of additional students.\n");
        freeMatrix(attendance, students);
        return 1;
    }

    if (additional > 0) {
        int oldCount = students;
        attendance = addStudents(attendance, oldCount, additional, days);
        students += additional;
        recordAttendance(attendance, oldCount, students, days);

        printf("\nAttendance Matrix After Dynamic Resizing:\n");
        displayAttendance(attendance, students, days);
    }

    calculatePercentages(attendance, students, days);
    freeMatrix(attendance, students);
    return 0;
}
```

Explanation

The matrix is represented by int **attendance. The first allocation creates an array of pointers, one for each student. Each pointer then refers to a separately allocated row containing the attendance for all working days.

When more students are added, realloc() enlarges the array of row pointers. New rows are then created with malloc(). A temporary pointer is used with realloc() so that the original pointer is not lost if reallocation fails.

Attendance input is restricted to 0 and 1. The percentage for a student is obtained by adding the row values (the number of present days), dividing by the number of working days, and multiplying by 100.

At the end, freeMatrix() releases every row followed by the main pointer array, preventing memory leaks.

Test Case / Sample Run

Student	Day 1	Day 2	Day 3	Attendance %
Student 1	1	0	0	33.33%
Student 2	1	0	1	66.67%
Student 3	1	1	1	100.00%
Student 4	1	0	0	33.33%
Student 5	1	1	1	100.00%
Student 6	1	0	1	66.67%

Sample Output

Initial Attendance Matrix:
Student Day 1 Day 2 Day 3
Student 1 1 0 0
Student 2 1 0 1
Student 3 1 1 1

Attendance Matrix After Dynamic Resizing:
Student 1 1 0 0
Student 2 1 0 1
Student 3 1 1 1
Student 4 1 0 0
Student 5 1 1 1
Student 6 1 0 1

Attendance Percentage:
Student 1: 33.33%
Student 2: 66.67%
Student 3: 100.00%
Student 4: 33.33%
Student 5: 100.00%
Student 6: 66.67%

Result

The program successfully manages student attendance using a dynamically resizable matrix. It uses pointers, malloc() and realloc() for dynamic memory management, validates attendance input,

supports addition of new students, calculates attendance percentages correctly, and releases all allocated memory before termination.